Compte Rendu
Cybersécurité_Hachage

Exercice 1:

Dans cet exercice, nous avons cherché à calculer l'empreinte SHA-1 d'une image afin de comprendre le principe des fonctions de hachage et leur utilisation pour vérifier l'intégrité d'un fichier.

En premier, j'ai récupéré la première image depuis l'énoncé, la deuxième n'était pas visible donc j'ai récupéré son lien puis il a été téléchargée par la commande wget.

Après les images en été placé dans les téléchargements :
cd ~/Téléchargements

Télécharger la deuxième image à l'aide de la commande:
wget http://www.u-picardie.fr/ressources/web-EIJV-blanc.png

J'ai calculé les empreintes pour les deux images:
sha1sum image1.png

```
ghizlane@ghizlane-VirtualBox:~/Téléchargements$ sha1sum image1.png
44b5e92f25d2a372b0b7e161897645580f8d6bd3  image1.png
```

sha1sum web-EIJV-blanc.png

```
ghizlane@ghizlane-VirtualBox:~/Téléchargements$ sha1sum web-EIJV-blanc.png
3d91d20e6801bfb450c0fbcc39750ee845b44515  web-EIJV-blanc.png
```

conclusion:

Comme les deux empreintes sont différentes, cela signifie que les deux fichiers ne sont pas identiques. Même si les images peuvent sembler similaires visuellement, leur contenu binaire est différent.

Exercice 2:

Dans cet exercice, nous avons cherché à retrouver le message correspondant à l'empreinte MD5 suivante : `21232f297a57a5a743894a0e4a801fc3`

21232f297a57a5a743894a0e4a801fc3 : admin

Cela montre que les mots de passe simples peuvent être facilement retrouvés, car leurs empreintes sont souvent déjà enregistrées dans des bases de données.

L'intérêt du "salt" est d'ajouter une valeur aléatoire au mot de passe avant le hachage. Cela permet d'obtenir une empreinte différente même pour un même mot de passe, ce qui empêche l'utilisation de bases de données existantes et renforce la sécurité.

Exercice 3:

Dans cet exercice, nous avons créé un fichier nommé `htpasswd` contenant un mot de passe chiffré: toto:$apr1$jx3BGTnD$GRMgqnIyS0Au1T5B9rtMr1

Ensuite, nous avons utilisé l'outil **John the Ripper** avec la commande `john htpasswd` afin de retrouver le mot de passe à partir du hash. Cet outil utilise une attaque par dictionnaire pour tester automatiquement des mots de passe courants.

Le mot de passe a été retrouvé rapidement, ce qui montre qu'il était faible.

```
ghizlane@ghizlane-VirtualBox:~$ nano htpasswd
ghizlane@ghizlane-VirtualBox:~$ john htpasswd
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-
long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-
opencl"
Use the "--format=md5crypt-opencl" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Cracked 1 password hash (is in /home/ghizlane/snap/john-the-ripper/694/.john/john.pot),
 use "--show"
No password hashes left to crack (see FAQ)
ghizlane@ghizlane-VirtualBox:~$ john --show htpasswd
toto:super

1 password hash cracked, 0 left
```

Exercice 4:

Dans cet exercice, l'objectif était de générer un dictionnaire de mots de passe contenant toutes les combinaisons possibles de 3 lettres minuscules à l'aide de la commande `crunch`.

Et puis utiliser l'outil John the Ripper avec ce dictionnaire pour tenter de retrouver le mot de passe associé au hash fourni. Cette méthode repose sur une attaque par force brute limitée à un ensemble de mots générés.

Cet exercice met en évidence qu'un mot de passe simple et court peut être facilement retrouvé, même s'il est haché. Il est donc essentiel d'utiliser des mots de passe plus longs et plus complexes.

Exercice 5:

Dans cet exercice, j'ai créé une chaîne de caractères sous la forme **"NOM Prénom"**. Ensuite, nous avons utilisé cette chaîne pour générer un mot de passe de type Linux en **SHA256** avec la commande `openssl passwd -5`.

Le cours précise que, dans le format Linux, la valeur `5` correspond à **SHA256**.

J'ai exécuté une deuxième fois la même commande avec la même chaîne. Le résultat obtenu n'est pas identique, car la commande génère automatiquement un salt aléatoire. Cela montre l'intérêt d'ajouter salt au mot de passe avant le hachage, ce qui produit une empreinte différente même si le mot de passe en clair reste le même.

